



University of Engineering and Technology Lahore

Department of Electrical Engineering

CEP Report

Designing a system that allows PC to communicate with FPGA
memory through UART

Submitted By:

Haiqua Ghaffar 2024-EE-53

Hamza Khalid 2024-EE-88

Abdul Rafay 2024-EE-91

Supervisor:

Mr. Rehan Naeem

Miss Mahnoor Farhat

June 12, 2026

Contents

1	Introduction	2
1.1	Problem Statement	2
1.2	Significance	2
1.3	Objectives	2
2	Background and Literature Review	3
3	System Specification and Design	3
3.1	System Overview	3
3.2	Specifications	4
3.2.1	Input Specifications	4
3.2.2	Output Specifications	4
3.2.3	Timing Constraints	4
3.2.4	Hardware Requirements	4
3.3	Design	4
3.3.1	FIFO	5
3.3.2	UART	5
3.3.3	UART Transmitter	6
3.3.4	Memory Controller	7
3.3.5	Complete System	9
3.4	Design Deviations	10
4	Implementation	10
4.1	HDL Code Structure	11
4.2	Key Design Decisions	11
4.3	Synthesis Results	11
5	Testing, Verification, and FPGA Results	12
5.1	Simulation Results	12
5.2	Test Cases	12
5.3	FPGA Board Results	12
5.4	Comparison	13
6	Conclusion	13
7	References	13

Abstract

A system was designed using UART, memory controller, FIFO which allows PC to communicate with FPGA memory through UART. Every module created is tested in Questa Sim and then implemented on FPGA. The key results of the CEP includes successful communication between PC and FPGA memory. Tested the system by first using the write command to write into the FPGA memory and then used the read command to read from the FPGA memory.

1 Introduction

1.1 Problem Statement

The problem that this design addresses is the lack of a direct and simple way for a PC to communicate with the internal memory of an FPGA. In many digital design and testing phases, there is a need to write data into or read data from the memory present inside an FPGA, but this cannot be done without a proper communication interface. To solve this problem, a UART-based memory controller is implemented on the Nexys A7 FPGA, which allows a host PC to send read and write commands over a serial UART connection operating at a baud rate of 115200 bps. The system receives command packets from the PC, determines whether the operation is a read or a write, and then accesses the 8-bit addressable on-chip memory accordingly. In the case of a read operation, the data stored at the requested memory address is transmitted back to the PC, completing the two-way communication between the host and the FPGA memory.

1.2 Significance

The ability to communicate with FPGA internal memory from a PC is important in many real-world engineering applications. During the development and testing of digital systems, engineers often need to verify that data is being stored and retrieved correctly from on-chip memory, and without a communication interface this verification becomes very difficult. In embedded systems and data acquisition applications, there is frequently a need to transfer data between a host computer and an FPGA in real time, which requires a reliable and simple communication protocol. UART is one of the most widely used serial communication protocols due to its simplicity, low hardware requirements, and availability on almost all PCs and FPGAs, making it a practical choice for this purpose. Therefore, designing a system that bridges the PC and FPGA memory over UART not only solves a fundamental problem in digital design workflows but also provides a reusable and scalable solution that can be extended to more complex embedded applications in the future.

1.3 Objectives

- To design a synchronous FIFO buffer that manages data flow and provides flow control between communicating modules.
- To implement a UART capable of transmitting and receiving serial data bytes on an FPGA platform.

- To develop a memory controller that facilitates read and write operations to the onboard memory of an FPGA.

2 Background and Literature Review

The design of the proposed system is based on well-established principles of digital system design, particularly in the areas of UART communication, FIFO buffering, and memory-mapped controller architectures. Prior work in these domains has extensively demonstrated the effectiveness of modular hardware design using Finite State Machines (FSMs) for reliable control and data handling.

UART (Universal Asynchronous Receiver Transmitter) is a widely used serial communication protocol for asynchronous data exchange between digital systems. Standard UART implementations typically consist of separate transmitter and receiver modules with configurable baud rate generators. References such as Harris and Harris (2021) and Mano and Ciletti (2018) describe UART design as a combination of shift registers and control logic driven by sampling clocks, which formed the basis for the UART subsystem integration in this project.

FIFO (First-In First-Out) buffers are commonly used to decouple producer-consumer timing mismatches in digital systems. Prior implementations in literature highlight the importance of pointer-based memory management and full/empty flag generation to ensure data integrity. In this design, the FIFO was extended to support simultaneous read and write operations, improving throughput and aligning with real-time communication requirements.

Memory controller architectures based on FSMs are widely used in embedded and FPGA-based systems to manage structured access to memory through command-based interfaces. Existing literature emphasizes the use of staged state transitions to separate command decoding, address handling, and data transfer phases. This approach was adopted and further refined by introducing additional intermediate states to ensure correct FIFO interaction while maintaining a Moore machine structure.

Overall, the system integrates concepts from standard digital design methodologies, combining UART communication, FIFO buffering, and FSM-based memory control into a cohesive architecture suitable for FPGA implementation.

3 System Specification and Design

The final design consists of a UART-based memory access system implemented on the Nexys A7 FPGA. The system communicates with a host PC through a UART interface using a virtual COM port. Incoming commands are buffered in an RX FIFO and processed by a memory controller. The memory controller performs read and write operations on the memory module and sends responses back to the PC through a TX FIFO and UART transmitter.

3.1 System Overview

The system enables a user to interact with FPGA memory using a serial terminal application such as PuTTY. Commands transmitted from the PC are received by the UART receiver and temporarily stored in the RX FIFO. The memory controller reads commands

from the RX FIFO, decodes the requested operation, and performs the corresponding memory access. Read data and status responses are written into the TX FIFO and transmitted back to the PC through the UART transmitter. The RX and TX FIFOs decouple the UART communication rate from the memory controller processing rate, ensuring reliable data transfer and preventing data loss during communication.

3.2 Specifications

3.2.1 Input Specifications

- System clock frequency: 100 MHz
- Active-high reset signal
- UART serial receive input (`FPGA_SERIAL_RX`)
- UART baud rate: 115200 bps

3.2.2 Output Specifications

- UART serial transmit output (`FPGA_SERIAL_TX`)
- State indication through FPGA LEDs
- Memory read data transmitted back to the host PC

3.2.3 Timing Constraints

- System clock frequency: 100 MHz
- UART baud rate: 115200 bps
- FIFO operations synchronized to the system clock

3.2.4 Hardware Requirements

- Nexys A7-100T FPGA development board
- USB connection to host PC
- PuTTY serial terminal software
- AMD Vivado Design Suite

3.3 Design

The system was developed in a modular manner. The design process began with the implementation of a First-In-First-Out (FIFO) buffer, followed by the development of the UART communication interface. Since the UART receiver module was provided, the UART transmitter was designed and integrated with the receiver to form a complete UART subsystem. Finally, a memory controller was developed to interface the communication modules with the memory system.

3.3.1 FIFO

The FIFO is used as a temporary storage buffer between communicating modules operating at different rates. In the final design, separate FIFOs are used for receive (RX) and transmit (TX) data paths. The RX FIFO stores data received through UART before it is processed by the memory controller, while the TX FIFO stores outgoing data before transmission.

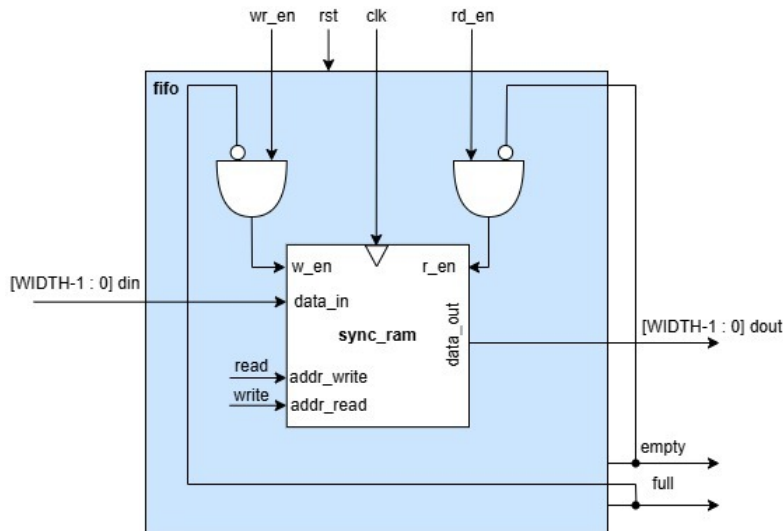


Figure 1: FIFO Block Diagram

Features

- Parameterized data width and depth.
- Separate read and write interfaces.
- Full and empty status indicators.
- Synchronous operation using the system clock.
- Protection against invalid read and write operations.

3.3.2 UART

The Universal Asynchronous Receiver Transmitter (UART) provides serial communication between the FPGA and the host PC. UART is an asynchronous communication protocol that does not require a shared clock between communicating devices. Instead, both transmitter and receiver operate using an agreed baud rate.

The UART subsystem used in this project consists of a UART receiver and UART transmitter. The receiver accepts serial data from the host PC while the transmitter sends responses back to the PC.

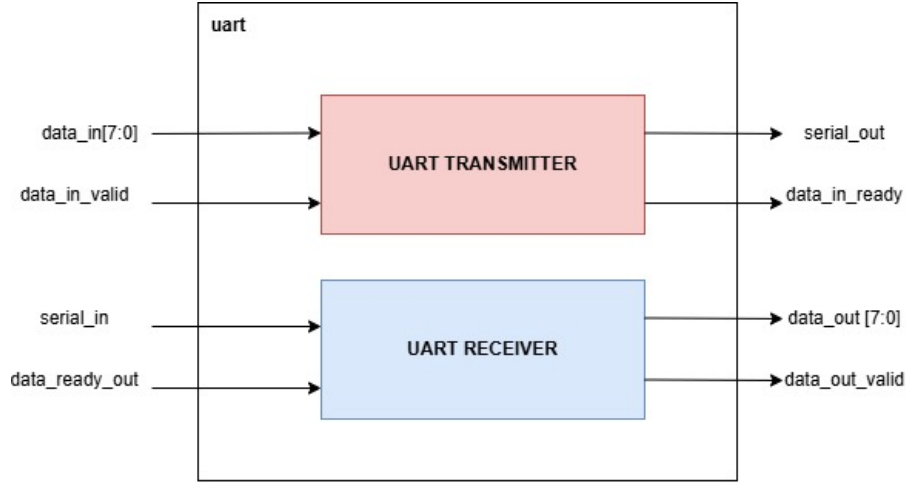


Figure 2: UART Block Diagram

3.3.3 UART Transmitter

The UART transmitter converts parallel data received through the ready-valid interface into a serial UART frame. Upon receiving a valid input byte, the transmitter generates a start bit, followed by eight data bits transmitted least-significant-bit first, and finally a stop bit. The transmission timing is controlled by a baud-rate counter derived from the system clock. Figure 3 shows the internal architecture of the UART transmitter. The design consists of an FSM Controller, a 10 bit shift register, a Clk Counter, a Bit Counter, Two comparators and a Combinational Latch which converts 8 bit data into 10 bit data (including the start and stop bits)

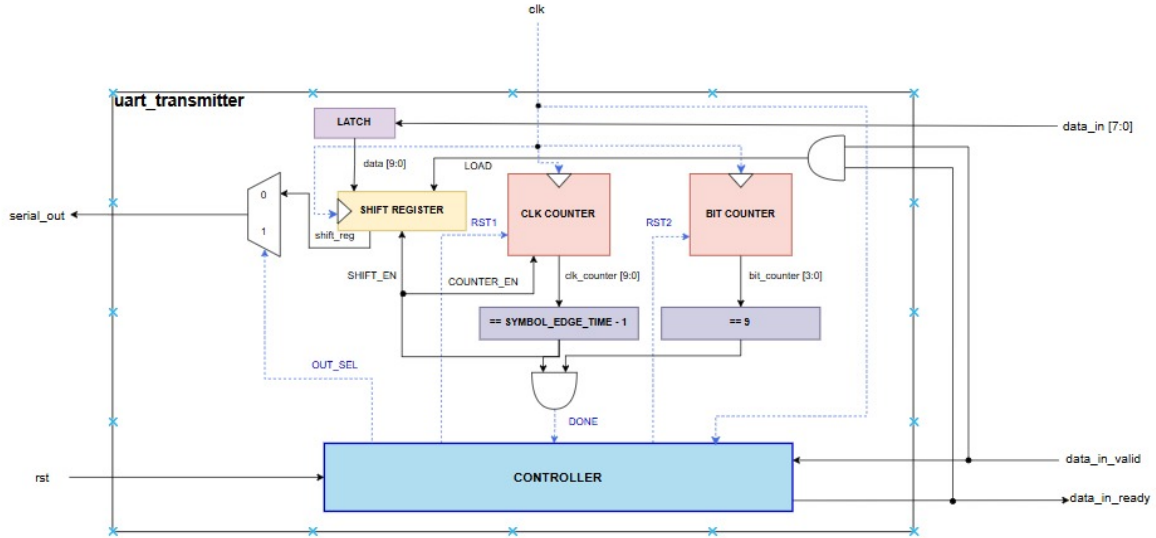


Figure 3: UART Transmitter Block Diagram

The operation of the UART transmitter is controlled by the Controller FSM shown in Figure 4. The FSM remains in the IDLE state until valid input data becomes available. It

then transmits the start bit, serializes the data bits, transmits the stop bit, and returns to the idle state. DONE signal from the datapath tells the controller that the shift register has successfully serialized all bits for Symbol Edge Time. The controller contains an Asynchronous reset signal, which when high, takes it back to IDLE state.

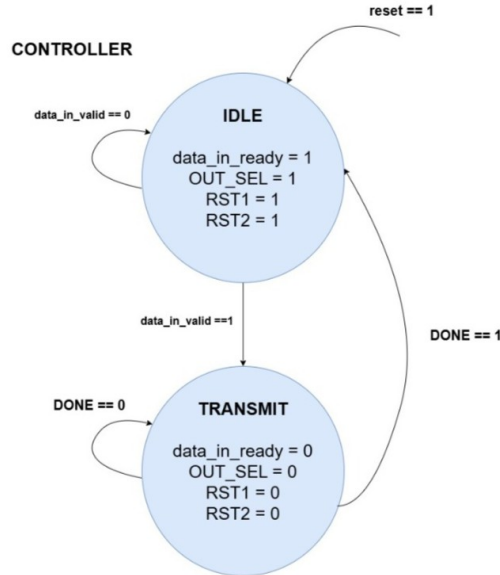


Figure 4: UART Transmitter State Transition Diagram

State Description

- **IDLE:** Waits for valid signal to go high, keeping the serial output high.
- **TRANSMIT:** Shifts the 10-bit data out of the Shift Register on the serial line.

3.3.4 Memory Controller

The memory controller coordinates communication between the UART, FIFOs, and the memory module. It receives read and write packet from the RX FIFO, decodes the requested operation, performs the corresponding memory access, and generates response packets for transmission through the TX FIFO.

Read operation packet: {READ , ADDR} READ: 8'd48

Write operation packet: {WRITE, ADDR, DATA} WRITE: 8'd49

Format of data packets, write (ASCII '1' = 8'd49) is 1 on the keyboard, and read (ASCII '0' = 8'd48) is 0 on the keyboard

It utilizes the empty signal of RX FIFO to check when a byte is available in the FIFO and controls its read enable to read a byte from it. Similarly, it uses the full signal of TX FIFO to check when it is read to accept a byte and generates its write enable accordingly.

Figure 5 illustrates the architecture of the memory controller and its interfaces with the RX FIFO, TX FIFO, and memory module.

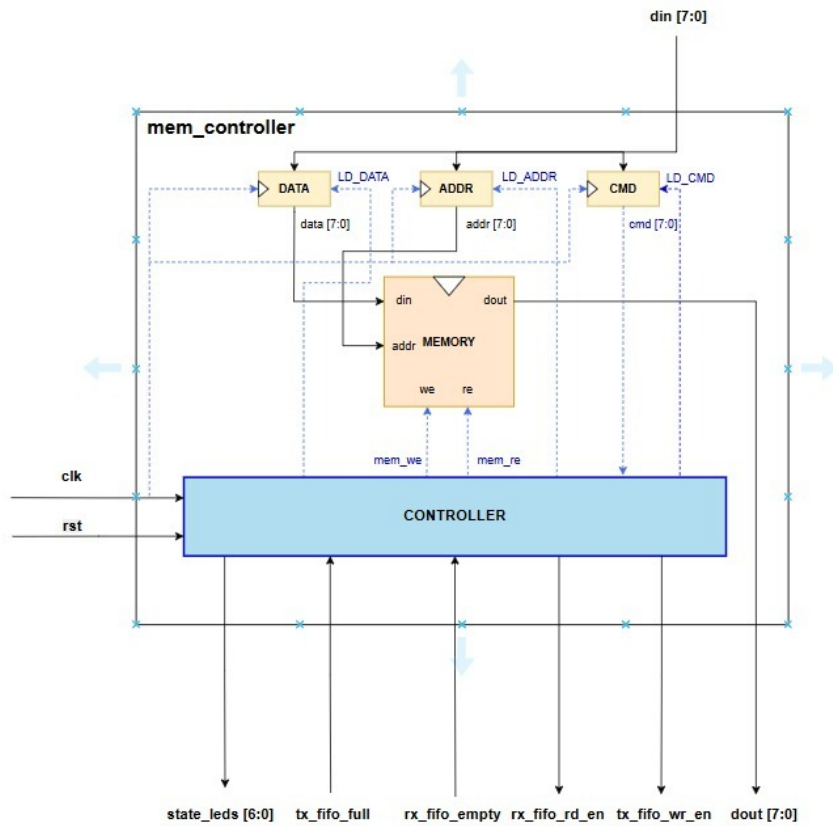


Figure 5: Memory Controller Block Diagram

The controller behavior is governed by the FSM shown in Figure 6. The FSM receives command bytes, decodes the requested operation, accesses memory, and generates the appropriate response packet.

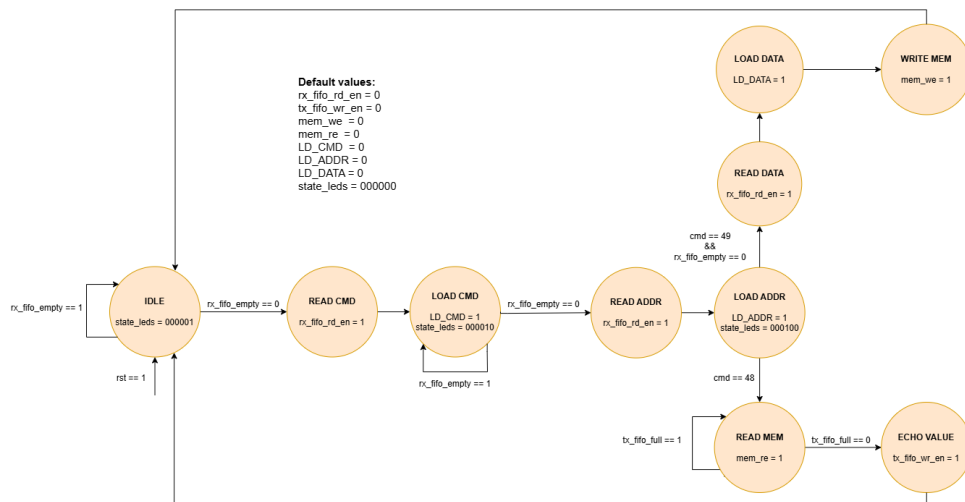


Figure 6: Memory Controller State Transition Diagram

State Description

The specific states depend on your implementation, but typically include:

- **IDLE**: Waits for data to become available in the RX FIFO. If data is present, transitions to command reception.
- **READ_CMD**: Initiates a read from the RX FIFO to fetch the command byte.
- **LOAD_CMD**: Latches the received command into the CMD register once valid data is available.
- **READ_ADDR**: Initiates a read from the RX FIFO to fetch the address byte.
- **LOAD_ADDR**: Latches the address and decodes the command to determine the operation (read or write).
- **READ_MEM**: Enables memory read operation; waits if the TX FIFO is full before proceeding.
- **ECHO_VALUE**: Writes the read memory value into the TX FIFO as a response, then returns to IDLE.
- **READ_DATA**: Initiates a read from the RX FIFO to fetch the data byte for a write operation.
- **LOAD_DATA**: Latches the received data byte for writing into memory.
- **WRITE_MEM**: Enables memory write operation by asserting write enable and then returns to IDLE.

3.3.5 Complete System

Putting everything together, figure 7 illustrates the Complete Architecture of the System and some extra logic to translate the read-valid interface of UART to read and write enable signals of TX and RX FIFOs.

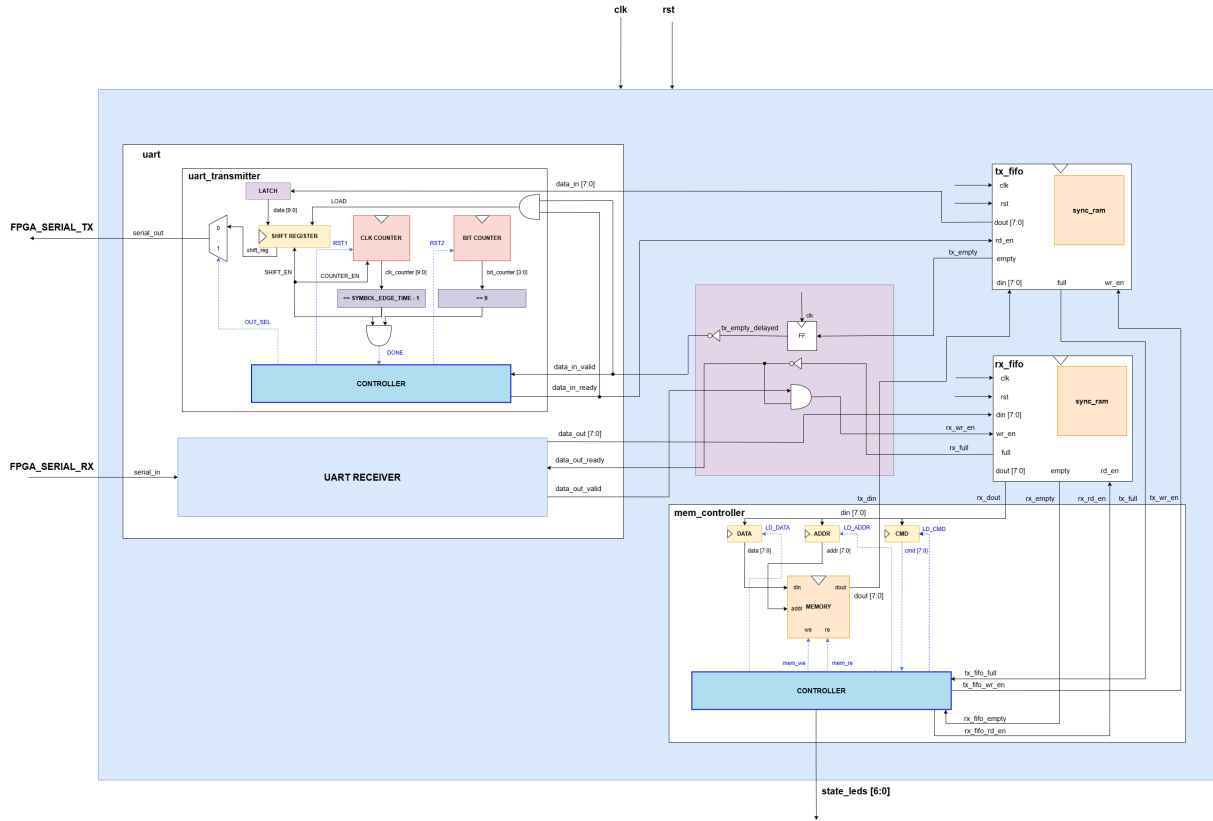


Figure 7: Block Diagram of the Complete System

3.4 Design Deviations

Several deviations were made from the original high-level design to improve correctness, simplicity, and modularity of the system. Firstly, additional intermediate states were introduced in the memory controller finite state machine (FSM). These extra states were required to ensure proper sequencing of FIFO read operations and reliable latching of incoming command, address, and data bytes. This also helped maintain the controller as a pure Moore machine, where outputs depend only on the current state, improving predictability and simplifying debugging. Secondly, a self-developed memory module with a single address port was used instead of a more complex dual-port or vendor-specific memory architecture. This choice was made because it satisfies the system requirements while remaining easier to understand, verify, and integrate within the overall design. Finally, support for simultaneous read and write operations was added to the FIFO design. This enhancement was necessary to meet system-level throughput requirements and to ensure smooth data flow between the UART interface and memory controller without introducing stalls or bottlenecks.

4 Implementation

The system was implemented in SystemVerilog using a modular design approach. The design follows a top-down hierarchy where communication, buffering, and memory control are separated into independent modules. This improves readability, debugging, and scalability. The complete system integrates UART communication, FIFO buffering, and

a memory controller on the Nexys A7 FPGA platform. The host PC communicates with the FPGA using UART, and all data transactions are managed through a controlled pipeline.

4.1 HDL Code Structure

The design follows a hierarchical module organization as shown below:

```
a7top
|-- uart
|   |-- uart_transmitter
|   |   |-- datapath_uart
|   |   '-- controller_uart
|   '-- uart_receiver
|-- tx_fifo
|   '-- sync_ram
|-- rx_fifo
|   '-- sync_ram
'-- mem_controller
    |-- memory
    |-- datapath_mem_ctrl
    '-- controller_mem_ctrl
```

Each module has a specific responsibility: UART handles serial communication, FIFOs manage data buffering, and the memory controller executes read/write operations.

4.2 Key Design Decisions

- **Memory Controller State Machine Design:** The memory controller was implemented using a finite state machine to manage read and write operations in a structured manner. Additional states were introduced to ensure proper sequencing of command decoding, memory access, and response generation. This expanded state machine allows reliable multi-cycle operation and correct synchronization between command input and memory execution.
- **UART to FIFO Interface Architecture:** A buffered interface was implemented between the UART module and the internal processing logic using RX and TX FIFOs. The RX FIFO stores incoming serial data after conversion to parallel form, while the TX FIFO holds outgoing data before transmission. This architecture ensures smooth data flow between asynchronous UART communication and synchronous system processing, while maintaining data integrity and proper sequencing.

4.3 Synthesis Results

The design was synthesized using Xilinx Vivado for the Nexys A7 FPGA board. The synthesis results confirmed correct hardware mapping of all modules.

- **Resource Utilization:** The design utilizes Look-Up Tables (LUTs) for combinational and control logic, flip-flops for storing sequential elements such as states and registers, and synchronous RAM-based structures for implementing FIFO buffers and

the memory module. These resources are efficiently mapped onto the FPGA fabric during synthesis.

- **Timing Analysis:** Timing analysis showed that the design successfully met the required clock constraints for UART operation at 115200 baud rate. No critical timing violations were observed, ensuring stable communication between FPGA and host PC.

5 Testing, Verification, and FPGA Results

The system was verified through simulation and hardware testing. All modules were first tested individually, followed by full system integration testing.

5.1 Simulation Results

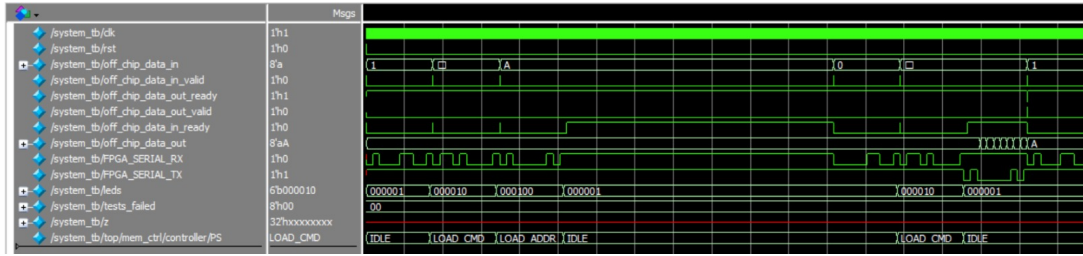


Figure 8: Simulation Waveform

5.2 Test Cases

Test Case	Expected Output	Result (Pass/Fail)
Memory Write Operation	Data stored correctly	Pass
Memory Read Operation	Correct data returned	Pass
UART Transmission	Correct byte received	Pass

Table 1: Test Case Results

5.3 FPGA Board Results

The design was implemented on the Nexys A7 FPGA using Xilinx Vivado. The system was tested using a PC connected via USB-UART interface.

PuTTY terminal was used to send read and write commands. The FPGA successfully executed all commands and returned correct responses through UART. This confirmed proper hardware functionality.

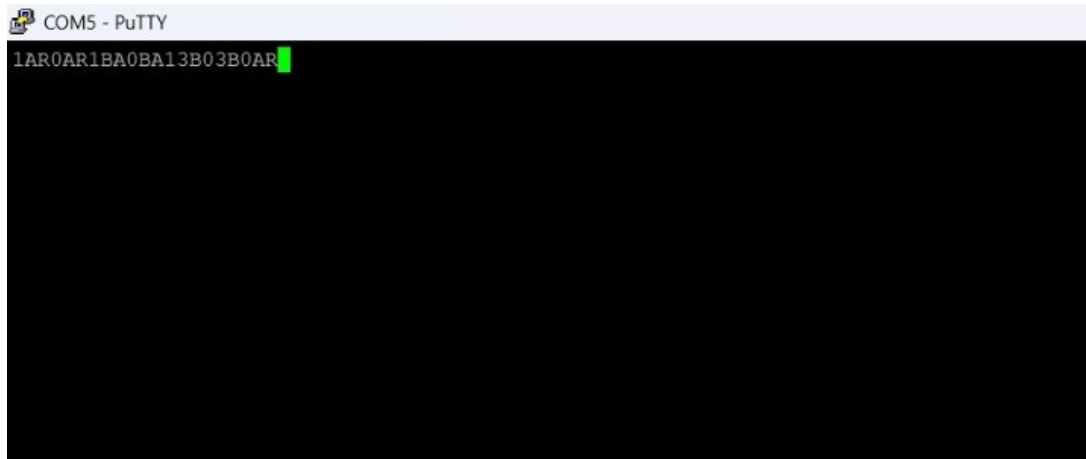


Figure 9: PuTTY Terminal Showing Memory Read and Write Operations on the FPGA

5.4 Comparison

Simulation and hardware results were consistent. Both environments showed correct data transmission and memory operations.

Minor differences in timing behavior were observed due to real-world UART communication delays, but no functional discrepancies were found.

6 Conclusion

The project successfully demonstrates the design and implementation of a modular digital system consisting of a UART interface, FIFO buffers, and an FSM-based memory controller. The system achieves reliable serial communication, correct command decoding, and proper read/write access to memory through a structured control architecture. PuTTY was used as a serial terminal interface to communicate with the FPGA system via UART. It enabled transmission of command, address, and data bytes to the system and allowed verification of memory read and write operations through received responses.

7 References

References

- [1] University of California, Berkeley, *FPGA Labs*, Department of Electrical Engineering and Computer Sciences, Berkeley, CA, USA, 2024.
- [2] S. D. Harris and D. M. Harris, *Digital Design and Computer Architecture*, 2nd ed., Morgan Kaufmann, 2012.